OBS STUDIO

Minimal Native Plugin Development Bundle

Windows x64 | C++17 | Visual Studio 2022 | OBS Studio 32.2.1

Reference implementation: OBS Orientation Switcher. A practical, reusable procedure for compiling a native OBS plugin with a locally maintained header and import-library set, without rebuilding the entire OBS Studio application.

Verification boundary: The referenced project successfully produced a Release x64 DLL with zero reported warnings and errors. Runtime loading and menu-command behaviour have not yet been verified in OBS.

DOCUMENT CONTROL

Edition: 1.0 | 6 November 2026

Scope: Windows desktop / MSVC / OBS 32.2.1

Status: Build procedure verified on the referenced project; deployment and runtime testing pending

Use: Internal development guide and reusable starter reference

1. Executive overview

The method separates building OBS itself from building a plugin against OBS. A plugin only needs the headers that define the APIs it uses, compatible import libraries for linking, and a suitable C++ build configuration. It does not need to rebuild OBS Studio for every plugin change.

This is a deliberately version-specific workflow. It is not a claim that every OBS plugin can use the same minimal header set, nor that DLL exports alone guarantee ABI compatibility.

2. Reference environment

Component	Reference configuration
Target OBS	OBS Studio 32.2.1, Windows x64
IDE / compiler	Visual Studio 2022; MSVC v143 / C++17
Plugin root	E:\VSCODE\obs-orientation-switcher
OBS source reference	E:\obs-studio-32.2.1
Installed OBS	C:\Program Files\obs-studio
Build target	Release | x64; Windows DLL

3. Suggested project layout

```
obs-orientation-switcher/
  obs-orientation-switcher.sln
  obs-orientation-switcher.vcxproj
  OBSPaths.props
  src/
    plugin-main.cpp
  lib/
    obs.h
    obs-module.h
    obs-frontend-api.h
    obs-config.h
    obsconfig.h
    obs.lib
    obs-frontend-api.lib
    graphics/  media-io/  util/  callback/
  tools/
    msvc/             (optional local build utilities)
  build/
    x64/Release/
```

Keep the SDK files under lib only for development. Do not deploy OBS runtime DLLs or PDBs with the plugin merely because they were copied into the working SDK folder.

4. Collecting a minimal local SDK

- Obtain public OBS headers from the same OBS release as the intended runtime. Copy the directly included headers and their transitive header dependencies.

- Preserve relative subdirectories such as callback, graphics, media-io and util; include statements inside OBS headers rely on those paths.
- When the compiler reports a missing header, identify the matching file in the release source tree and copy it into the corresponding location. Do not fabricate arbitrary replacements for API headers.
- Some headers are generated by the OBS build. In this reference case, obsconfig.h was required separately from obs-config.h.

The two similarly named configuration headers

```
lib\obs-config.h     // Existing OBS header; includes obsconfig.h
lib\obsconfig.h      // Separate, generated build-configuration header
```

The OBS source tree contained libobs/obsconfig.h.in, a CMake input template. For this specific Windows plugin build, a minimal local generated header was sufficient:

```
#pragma once
#define OBS_RELEASE_CANDIDATE 0
#define OBS_BETA 0
```

Important: These values describe the referenced stable build. For other OBS releases, platforms, or feature-dependent headers, generate the configuration using the corresponding OBS build configuration rather than assuming this substitute is sufficient.

5. Import libraries: what they are and how to create them

Windows DLL files expose runtime functions, while MSVC normally links against an import library (.lib). The installed OBS directory may include obs.dll and obs-frontend-api.dll without the corresponding development import libraries.

The reference workflow inspected named DLL exports with MSVC dumpbin.exe, produced module-definition (.def) files, then generated x64 import libraries with lib.exe. The tools were run from a complete MSVC tools directory or properly configured Developer Command Prompt.

Illustrative commands

```
dumpbin /exports "C:\Program Files\obs-studio\bin\64bit\obs.dll"
dumpbin /exports "C:\Program Files\obs-studio\bin\64bit\obs-frontend-api.dll"

lib /machine:x64 /def:obs.def /out:obs.lib
lib /machine:x64 /def:obs-frontend-api.def /out:obs-frontend-api.lib
```

A .def file needs the correct module name and actual exported names. Example structure:

```
LIBRARY obs.dll
EXPORTS
    obs_get_version_string
```

The single-symbol example above is not a complete obs.def. In a working project, populate EXPORTS from the matching DLL export table. Avoid including ordinal-only entries as invented function names. Validate that all referenced symbols link successfully.

Limitations: A generated import library only solves symbol lookup at link time. It does not validate header/DLL ABI agreement, OBS plugin compatibility, calling conventions, or runtime dependencies. Prefer official build artifacts when available and suitable.

6. MSBuild property sheet

The following is the working baseline used for the successful build. The OBS include directory and both import libraries are project-local.

```
<?xml version="1.0" encoding="utf-8"?>
<Project xmlns="http://schemas.microsoft.com/developer/msbuild/2003">
  <PropertyGroup>
    <OBSSDKDir>$(MSBuildThisFileDirectory)lib\</OBSSDKDir>
  </PropertyGroup>
  <ItemDefinitionGroup>
    <ClCompile>

<AdditionalIncludeDirectories>$(OBSSDKDir);%(AdditionalIncludeDirectories)</AdditionalIncludeDirectori
es>
      <WarningLevel>Level4</WarningLevel>
    </ClCompile>
    <Link>

<AdditionalLibraryDirectories>$(OBSSDKDir);%(AdditionalLibraryDirectories)</AdditionalLibraryDirectori
es>

<AdditionalDependencies>obs.lib;obs-frontend-api.lib;%(AdditionalDependencies)</AdditionalDependencies
>
    </Link>
  </ItemDefinitionGroup>
</Project>
```

Import OBSPaths.props into the intended project configurations through the Visual Studio Property Manager or the .vcxproj import structure. Confirm that the Release x64 build actually imports it.

7. Handling C4201 warnings correctly

OBS public headers use unnamed structures/unions in places. MSVC may issue C4201 under warning level 4. These are header-origin diagnostics, not evidence of a defect in the plugin source.

The tested approach was to suppress only C4201 while processing the OBS headers:

```
#pragma warning(push)
#pragma warning(disable : 4201)

#include <obs-module.h>
#include <obs-frontend-api.h>

#pragma warning(pop)
```

Place all other OBS header includes that trigger C4201 inside the same guarded block. This preserves compiler diagnostics for the plugin implementation outside the include region.

Failed experiment: Replacing the working include setup with an /external:I command-line configuration led to MSVC D8003: missing source filename in this project. The property sheet was restored and the targeted pragma solution built successfully. D8003 alone does not prove which exact command-line token was malformed.

8. Build configuration and verification

- Select Release | x64. Confirm that the project configuration type is Dynamic Library (.dll).
- Use a consistent MSVC toolset and architecture for both import-library generation and compilation.
- Build the solution. If a configuration change appears not to take effect, perform Clean Solution and then Rebuild Solution.

- Verify that the DLL exists in the configured output directory. A successful build is necessary but not sufficient for successful OBS loading.

Verified result from the reference project

```
obs-orientation-switcher.vcxproj ->
E:\VSCODE\obs-orientation-switcher\build\x64\Release\obs-orientation-switcher.dll

Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped
```

The final log reported no warnings. The earlier linker messages about generating .lib and .exp files were normal DLL build output, not errors.

9. Deployment and runtime validation

Close OBS before copying the DLL. For a conventional Windows installation, the OBS plugin binary directory is:

```
C:\Program Files\obs-studio\obs-plugins\64bit\
```

Copy only the built plugin DLL and any genuinely required plugin-owned runtime dependencies or data files. Avoid overwriting OBS core DLLs. Administrator permissions may be needed for Program Files.

Runtime verification must be performed in OBS Studio itself:

- Launch the intended OBS 32.2.1 installation and confirm the plugin loads; inspect Help > Log Files > View Current Log for module load errors.
- Confirm both Tools-menu entries appear.
- Use a duplicate OBS profile; stop recording, streaming and other active outputs before changing video settings.
- Check Settings > Video after each action: portrait should set base and output to 1080 x 1920; landscape should set both to 1920 x 1080.
- Test switching in both directions and restarting OBS. Verify scene framing, output encoder compatibility and any downstream integrations.

Changing the video resolution does not automatically guarantee scene items will be reframed. Scene transforms and canvas composition may need separate handling in a later version.

10. Troubleshooting matrix

Symptom	Likely cause / next action
C1083: callback/signal.h	Missing transitive header; copy matching callback tree.
C1083: obs-config.h	Missing OBS configuration header; copy from matching source.
C1083: obsconfig.h	Separate generated configuration header missing.
C4201 warnings	Guard OBS includes with push/disable 4201/pop.
D8003 missing source filename	Inspect full cl.exe command; revert malformed AdditionalOptions.
LNK2019 unresolved external	Check .lib architecture, exports, function signature and linker inputs.
DLL builds but does not load	Inspect OBS log, module exports, runtime dependencies and version match.
Tools menu absent	Confirm module loaded and registration code executed.

11. Reuse checklist for a new plugin

- Create a new Visual Studio C++ DLL project targeting x64 and C++17.
- Copy the local, version-matched OBS header/import-library SDK folder or point the new project to a shared, versioned SDK location.
- Import OBSPaths.props; ensure the expected .lib files are present.
- Add OBS module entry points and any required frontend integration to the plugin source.
- Guard OBS headers against C4201, without muting unrelated project warnings.
- Build Release x64 and confirm a clean compiler/linker result.
- Install into a test OBS environment and inspect the runtime log.
- Validate every advertised plugin function and document tested OBS versions.

12. Engineering cautions and maintenance

- Version pinning: Keep SDK folders clearly labelled by OBS version. Revalidate the build and runtime behaviour when upgrading OBS.
- Header integrity: Avoid editing upstream API declarations. Maintain generated config headers separately and document why they exist.
- Licensing: Respect applicable OBS and dependency licence requirements when distributing code, binaries or copied SDK headers. Review the specific licences before public distribution.
- Reproducibility: Retain the build configuration, import-library generation procedure, and an inventory of SDK headers; do not depend on an undocumented personal machine state.
- Security and stability: Validate user-visible actions at runtime. A native plugin executes in the OBS process and can crash it if incompatible.

Appendix A. Verified vs. pending

Verified: Local OBS SDK header inclusion; local x64 import-library linking; generated obsconfig.h accepted by the compiler; targeted C4201 suppression; successful Release x64 DLL creation with no reported warnings/errors.

Not yet verified: OBS runtime loading, menu item visibility, orientation changes, persistence across restarts, scene layout behaviour, and compatibility with other OBS releases.

Appendix B. Reference commands

```
REM Developer Command Prompt / appropriate privileges as required
REM Inspect exports:
dumpbin /exports "C:\Program Files\obs-studio\bin\64bit\obs.dll"

REM Rebuild from Visual Studio after modifying properties:
REM Build > Clean Solution
REM Build > Rebuild Solution

REM Install the plugin only after closing OBS:
copy /Y "E:\VSCODE\obs-orientation-switcher\build\x64\Release\obs-orientation-switcher.dll" "C:\
Program Files\obs-studio\obs-plugins\64bit\obs-orientation-switcher.dll"
```

End of guide. This is a reproducible development procedure for the referenced setup, not a substitute for runtime compatibility testing or the official OBS API documentation.